

THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

Data Science Program

Capstone Report - Fall 2025

Reinforcement Learning for Pseudo-Labeling in Semi-Supervised Learning

Aishwarya Maddula, Deepika Reddygari
Prudhvi Chekuri, Phanindra Kalaga

supervised by
Prof. Tyler Wallett
Dr. Amir Jafari

Abstract

0.7 Semi-supervised learning leverages abundant unlabeled data to improve performance when labeled data is scarce. This project develops a reinforcement learning (RL) framework for intelligent pseudo-labeling, where an RL agent learns to assign high-quality pseudo-labels to unlabeled samples. We implement and evaluate a Policy Gradient approach on MNIST and a Reinforcement Learning-Guided Semi-Supervised Learning (RLGSSL) method inspired by recent research. Our results demonstrate that confidence-based rewards and adaptive training can improve pseudo-labeling quality, though challenges remain in computational complexity and reward design. These findings provide insights for future work in RL-guided semi-supervised learning.

Contents

1. Introduction **4**
- 1.1. Project Objectives . . . 4
- 1.2. Project Scope . . . 4

2. Problem Statement **6**
- 2.1. Semi-Supervised Learning Challenges . . . 6
- 2.2. Reinforcement Learning Formulation . . . 6
- 2.3. Technical Challenges . . . 6

3. Related Work **7**
- 3.1. Semi-Supervised Learning . . . 7
- 3.2. Pseudo-Labeling Techniques . . . 7
- 3.3. Reinforcement Learning-Guided Semi-Supervised Learning . . . 8
- 3.4. Reinforcement Learning for Data Selection . . . 8
- 3.5. Comparison with Our Approach . . . 8

4. Solution and Methodology **9**
- 4.1. Approach 1: Custom Policy Gradient Framework . . . 9
  - 4.1.1. Environment Design . . . 9
  - 4.1.2. Policy Gradient Agent . . . 10
  - 4.1.3. Downstream Model . . . 10
- 4.2. Approach 2: RLGSSL Framework . . . 11
  - 4.2.1. Teacher-Student Framework . . . 11
  - 4.2.2. Mixup Data Generation . . . 11
  - 4.2.3. RL Reward Function . . . 11
  - 4.2.4. RL Loss Function . . . 12
  - 4.2.5. Combined Loss Function . . . 12
- 4.3. Implementation Details . . . 12

5. Results and Discussion **13**
- 5.1. Experimental Setup . . . 13
  - 5.1.1. Dataset Configuration . . . 13
  - 5.1.2. Model Architectures . . . 13
  - 5.1.3. Hyperparameters . . . 14
- 5.2. Policy Gradient Results . . . 14
- 5.3. RLGSSL Results . . . 15
  - 5.3.1. Training Progression . . . 15
  - 5.3.2. Comparison with Literature . . . 15
- 5.4. Ablation Studies . . . 16
- 5.5. Computational Analysis . . . 16
- 5.6. Analysis of Results . . . 16
  - 5.6.1. Strengths Observed . . . 16
  - 5.6.2. Limitations and Challenges . . . 17

6. Discussion **17**
- 6.1. Technical Challenges . . . 17
  - 6.1.1. Reward Function Design . . . 17
  - 6.1.2. Computational Constraints . . . 17
  - 6.1.3. Non-Stationarity and Credit Assignment . . . 18
- 6.2. Insights on RLGSSL Formulation . . . 18
  - 6.2.1. Policy Network as Classifier . . . 18

6.2.2. Mixup as Environment Dynamics . . . 18
6.3. Comparison: Policy Gradient vs. RLGSSL . . . 19
6.4. Practical Considerations . . . 19
6.4.1. When to Use RL for Pseudo-Labeling . . . 19
6.4.2. Engineering Recommendations . . . 19
6.5. Future Directions . . . 20
6.6. Limitations of Current Work . . . 20

7. Conclusion **20**
7.1. Key Contributions . . . 21
7.2. Main Findings . . . 21
7.3. Lessons Learned . . . 21
7.4. Broader Impact . . . 22
7.5. Future Work . . . 22
7.6. Closing Remarks . . . 22

A. Appendix **24**
A.1. A: Code Structure . . . 24
A.2. B: Hyperparameter Configuration . . . 24
A.3. C: Key Implementation Details . . . 25
A.4. D: References for Additional Reading . . . 25
A.5. E: Acknowledgments . . . 26

1. Introduction

In modern machine learning applications, obtaining large quantities of labeled training data is often expensive, time-consuming, and requires domain expertise. Semi-supervised learning (SSL) offers a promising solution by leveraging both limited labeled data and abundant unlabeled data to improve model performance. A key technique in SSL is pseudo-labeling, where a model assigns estimated labels to unlabeled samples, which are then used for further training.

Traditional pseudo-labeling approaches rely on simple heuristics such as confidence thresholds, often selecting pseudo-labels based solely on prediction confidence. However, these methods can propagate errors when incorrect high-confidence predictions are blindly accepted, leading to confirmation bias and degraded model performance.

This project addresses these limitations by framing pseudo-labeling as a reinforcement learning problem. By treating the selection and assignment of pseudo-labels as a sequential decision-making task, we enable an intelligent agent to learn optimal labeling strategies that balance exploration (diverse samples) and exploitation (high-confidence samples).

1.1. Project Objectives

The primary objectives of this capstone project are:

1. **RL Framework Development**: Design and implement a reinforcement learning framework that can automatically assign pseudo-labels to unlabeled data in a principled manner.

2. **Custom Environment Design**: Build a custom OpenAI Gymnasium environment specifically tailored for the pseudo-labeling task, with carefully designed state spaces, action spaces, and reward functions.

3. **Policy Gradient Implementation**: Implement and evaluate Policy Gradient methods (Actor-Critic) for learning pseudo-labeling strategies.

4. **RLGSSL Exploration**: Investigate and implement the Reinforcement Learning-Guided Semi-Supervised Learning approach proposed by Heidari et al. [1], which integrates RL with teacher-student frameworks and mixup augmentation.

5. **Empirical Evaluation**: Conduct comprehensive experiments on standard datasets (MNIST, CIFAR-10) to evaluate the effectiveness of RL-based pseudo-labeling approaches.

1.2. Project Scope

This project explores two complementary approaches to RL-based pseudo-labeling:

Approach 1: Custom RL Environment with Policy Gradients – We developed a complete RL framework from scratch, including:

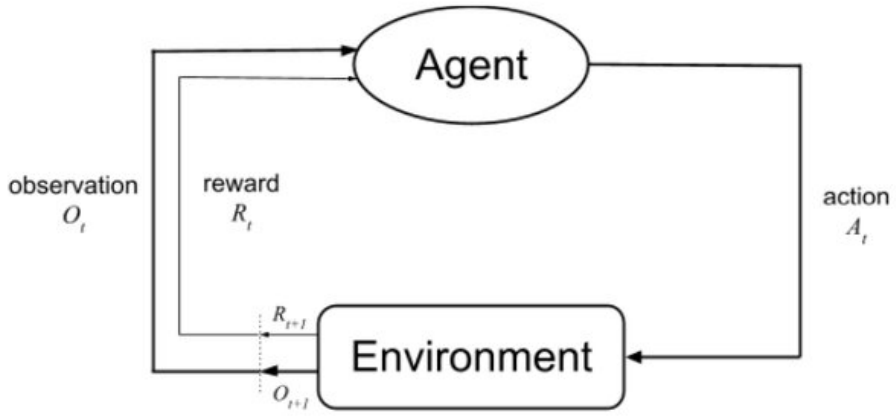


Figure 1: RL agent-environment loop. The agent selects actions and the environment responds and presents new observations and rewards to the agent again. The aim is for the agent to learn to maximize rewards through repeated interaction.

- A custom PseudoLabelEnv environment implementing the OpenAI Gymnasium interface
- State representation combining image features, softmax predictions, and prediction entropy
- Action space allowing the agent to assign pseudo-labels or skip uncertain samples
- Confidence-based reward functions that encourage correct, high-confidence predictions
- Policy Gradient (Actor-Critic) agent for learning optimal pseudo-labeling strategies

Approach 2: RLGSSL Framework – Building on recent research, we investigated the RLGSSL approach which integrates:

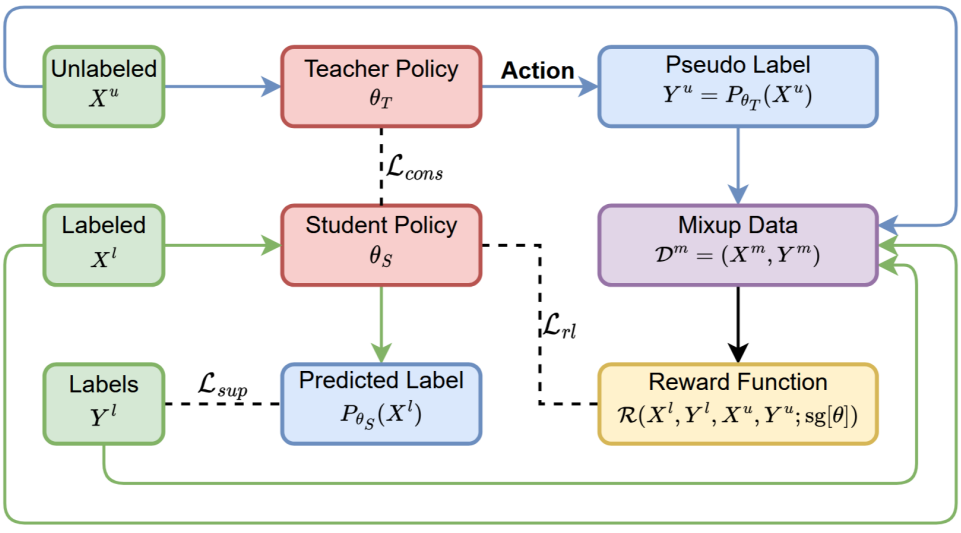


Figure 2: Overview of the RLGSSL Framework. The prediction networks (θ_S, θ_T) serve as the policy functions, and pseudo-labeling ($P_{\theta_T}(X^u)$) acts as the actions. The model has three loss terms in total: RL loss ($\mathcal{L}_{\text{rl}}$), supervised loss ($\mathcal{L}_{\text{sup}}$), and consistency loss ($\mathcal{L}_{\text{cons}}$). The teacher policy function is used to execute the actions and compute the consistency loss, while the student policy function is used for all other aspects.

- Teacher-student framework with Exponential Moving Average (EMA) updates

- Mixup data augmentation for generating synthetic training samples
- Novel RL reward function based on model performance on mixup samples
- Adaptive enhancements including confidence bonuses, diversity regularization, and curriculum learning

This report details our implementation methodology, experimental setup, results, challenges encountered, and insights gained throughout this capstone project.

2. Problem Statement

2.1. Semi-Supervised Learning Challenges

Semi-supervised learning aims to leverage unlabeled data to improve model performance when labeled data is scarce. While pseudo-labeling has shown promise as a simple yet effective SSL technique, several fundamental challenges remain:

1. **Confirmation Bias**: Traditional pseudo-labeling methods tend to reinforce existing model biases. When a model makes confident but incorrect predictions, using these as pseudo-labels amplifies errors through a feedback loop.
2. **Quality vs. Quantity Trade-off**: Aggressive pseudo-labeling (using low confidence thresholds) provides more training data but risks adding noisy labels. Conservative approaches (high confidence thresholds) ensure quality but fail to leverage the full potential of unlabeled data.
3. **Static Decision Rules**: Existing methods typically use fixed heuristics (e.g., "accept if confidence > 0.95") that do not adapt to the evolving model capabilities or dataset characteristics during training.
4. **Sample Selection Strategy**: Determining which unlabeled samples should be pseudo-labeled at each training iteration is non-trivial. Optimal selection requires balancing multiple factors including prediction confidence, sample diversity, and potential impact on model improvement.

2.2. Reinforcement Learning Formulation

To address these challenges, we formulate pseudo-labeling as a sequential decision-making problem suitable for reinforcement learning. This formulation enables:

- **Adaptive Strategies**: The RL agent learns to adjust its pseudo-labeling behavior based on accumulated experience and model evolution.
- **Multi-objective Optimization**: Through reward shaping, we can encode multiple objectives (accuracy, confidence, diversity) into a single optimization framework.
- **Exploration-Exploitation Balance**: The agent naturally balances exploring uncertain samples versus exploiting high-confidence predictions.

2.3. Technical Challenges

Applying reinforcement learning to pseudo-labeling introduces several technical challenges:

1. **State Space Design**: Determining effective feature representations that capture both image characteristics and model uncertainty.

2. **Reward Function Engineering**: Designing reward signals that encourage correct pseudo-labeling while avoiding reward hacking and maintaining training stability.

3. **Computational Complexity**: RL training typically requires many episodes and environment interactions, which can be computationally expensive when each episode involves training a downstream classification model.

4. **Non-Stationarity**: The underlying classification model evolves during training, violating the Markov assumption and introducing non-stationarity in the environment.

5. **Credit Assignment**: Determining which pseudo-labeling decisions contributed to final model performance is challenging when rewards are delayed until episode completion.

6. **Scalability**: Ensuring the approach scales to larger datasets and more complex architectures beyond simple benchmarks.

These challenges motivate our exploration of both custom RL frameworks and recent research directions like RLGSSL, which we detail in subsequent sections.

3. Related Work

Our work builds upon and contributes to several interconnected research areas in machine learning. This section positions our approach with respect to existing literature on semi-supervised learning, pseudo-labeling, and reinforcement learning applications.

3.1. Semi-Supervised Learning

Semi-supervised learning has been extensively studied as a paradigm for learning from limited labeled data. **[TODO: Add specific citations to foundational SSL papers]**. Key approaches include:

- **Consistency Regularization**: Methods like MixMatch, FixMatch, and UDA enforce consistency between predictions on different augmentations of the same input.
- **Self-Training and Pseudo-Labeling**: Pioneered by **[TODO: Add Yarowsky 1995, Scudder 1965 citations]**, these methods iteratively train models on pseudo-labeled data.
- **Graph-Based Methods**: Approaches that propagate labels through similarity graphs constructed from data manifolds.

Unlike these methods which use fixed heuristics for pseudo-label selection, our RL-based approach learns adaptive selection strategies.

3.2. Pseudo-Labeling Techniques

Traditional pseudo-labeling assigns labels to unlabeled samples based on model predictions exceeding a confidence threshold. Recent improvements include:

- **Confidence-Based Selection**: Methods that dynamically adjust confidence thresholds during training **[TODO: Add FixMatch citation (Sohn et al., 2020)]**.
- **Temporal Ensembling**: Using exponential moving averages of model parameters to produce more stable pseudo-labels **[TODO: Add Mean Teacher citation (Tarvainen & Valpola, 2017)]**.
- **MixUp Augmentation**: Interpolating between samples to create synthetic training examples with soft labels **[TODO: Add MixUp citation (Zhang et al., 2018)]**.

3.3. Reinforcement Learning-Guided Semi-Supervised Learning

Our primary inspiration comes from Heidari et al. [1], who proposed RLGSSL, a novel framework that formulates semi-supervised learning as a reinforcement learning problem. Their key contributions include:

- **RL Policy as Classifier**: The classification network itself serves as the RL policy, eliminating the need for separate actor-critic architectures.
- **Mixup-Based Rewards**: Reward signals derived from model performance on synthetically mixed samples, implemented with stop-gradient operations to prevent direct gradient flow.
- **Novel Loss Formulation**: An RL loss function based on KL divergence weighting: $\mathcal{L}_{rl} = -\mathbb{E}_{y_i^u \sim \pi_\theta}[\mathrm{KL}(e, y_i^u) \times R(s, a; \mathrm{sg}[\theta])]$
- **State-of-the-Art Results**: Achieved error rates of 9.15% on CIFAR-10 with 1000 labels using CNN-13 architecture.

Our implementation extends RLGSSL with adaptive enhancements including confidence bonuses, diversity regularization, and curriculum learning strategies that were not present in the original paper.

3.4. Reinforcement Learning for Data Selection

Several works have explored using RL for intelligent data selection in machine learning pipelines:

- **Active Learning**: RL agents that learn to query informative samples for labeling **[TODO: Add active learning RL citations]**.
- **Curriculum Learning**: Using RL to determine optimal ordering of training samples **[TODO: Add curriculum learning citations]**.
- **Data Augmentation**: Learning augmentation policies through reinforcement learning **[TODO: Add AutoAugment citation (Cubuk et al., 2019)]**.

3.5. Comparison with Our Approach

Our work differs from existing approaches in several key aspects:

1. **Dual Framework Implementation**: We explore both custom RL environments with explicit state-action formulations and integrated RLGSSL approaches, providing comparative insights.
2. **Adaptive Enhancements**: Beyond the base RLGSSL formulation, we introduce confidence bonuses, diversity regularization, and curriculum learning mechanisms.
3. **Comprehensive Engineering**: Our implementation includes complete training pipelines, custom Gymnasium environments, and systematic hyperparameter configurations.
4. **Practical Challenges Documentation**: We explicitly document computational challenges, stability issues, and practical considerations often omitted from research papers.

The next section details our solution methodology, including architecture designs, algorithm implementations, and the technical decisions underlying our framework.

4. Solution and Methodology

This section presents our comprehensive approach to RL-based pseudo-labeling. We describe two complementary implementations: a custom Policy Gradient framework and an RLGSSL-based approach with adaptive enhancements.

4.1. Approach 1: Custom Policy Gradient Framework

4.1.1. Environment Design

We implemented a custom OpenAI Gymnasium environment (`PseudoLabelEnv`) that encapsulates the pseudo-labeling task as a reinforcement learning problem.

State Space ($\mathcal{S}$): For each unlabeled sample x_i, the state s_i is constructed as:

$$s_i = [\phi(x_i), P_\theta(x_i), H(P_\theta(x_i))] \tag{1}$$

where:

- $\phi(x_i) \in \mathbb{R}^{784}$ are the flattened image features
- $P_\theta(x_i) \in \mathbb{R}^{10}$ are the softmax class probabilities from the downstream model
- $H(P_\theta(x_i)) = -\sum_k P_\theta(x_i)_k \log P_\theta(x_i)_k$ is the prediction entropy

This representation combines raw features, model confidence, and uncertainty, providing the agent with comprehensive information for decision-making.

Action Space ($\mathcal{A}$): The action space consists of $|\mathcal{A}| = K + 1$ discrete actions for a K-class classification problem:

$$\mathcal{A} = \{0, 1, \ldots, K-1, \text{skip}\} \tag{2}$$

Actions 0 through $K-1$ assign the corresponding class label as a pseudo-label, while the skip action ($a = K$) indicates the sample should not be pseudo-labeled.

Reward Function (R): We designed a confidence-weighted reward function:

$$R(s_i, a_i) = \begin{cases} -0.1 & \text{if } a_i = \text{skip} \\ P_\theta(x_i)_{a_i} & \text{if } a_i = y_i^{\text{true}} \\ -P_\theta(x_i)_{a_i} & \text{if } a_i \neq y_i^{\text{true}} \end{cases} \tag{3}$$

where y_i^{true} is the true label (used only for reward computation during training). This design encourages correct, high-confidence predictions while penalizing incorrect ones proportionally to the model's misplaced confidence.

Episode Structure: Each episode processes all N unlabeled samples sequentially. At episode completion, the downstream model is retrained on the combination of original labeled data and newly pseudo-labeled samples, and the agent receives an accuracy bonus:

$$R_{\text{final}} = \text{Acc}_{\text{new}} - \text{Acc}_{\text{baseline}} \tag{4}$$

This provides a delayed reward signal based on the cumulative impact of pseudo-labeling decisions.

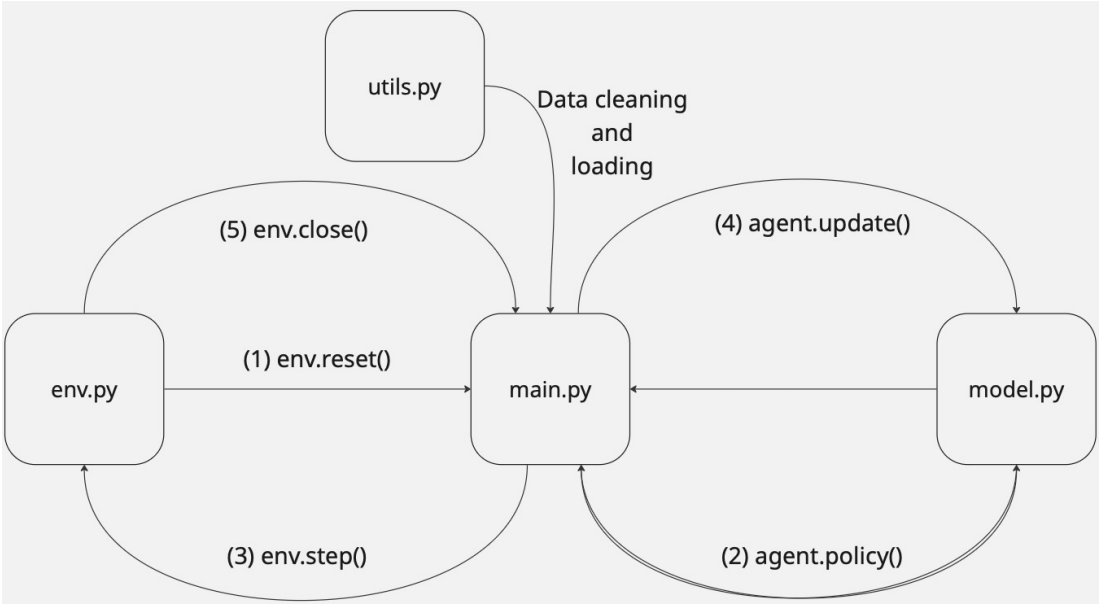


Figure 3: Environment flowchart: reset → get state → agent action → step → reward → repeat until terminated.

4.1.2. Policy Gradient Agent

We implemented an Actor-Critic Policy Gradient agent with the following architecture:

Policy Network (Actor): Maps states to action logits:

$$h_1 = \tanh(W_1 s + b_1) \tag{5}$$

$$h_2 = \tanh(W_2 h_1 + b_2) \tag{6}$$

$$\text{logits} = W_3 h_2 + b_3 \tag{7}$$

where hidden dimensions are 64 neurons per layer.

Value Network (Critic): Estimates state values $V(s)$ using a similar architecture but with scalar output.

Training Procedure: At the end of each episode, we compute:

- Rewards-to-go: $G_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$
- Advantage estimates: $A_t = G_t - V(s_t)$
- Policy loss: $\mathcal{L}_\pi = -\mathbb{E}_t[\log \pi_\theta(a_t|s_t) A_t]$
- Value loss: $\mathcal{L}_V = \mathbb{E}_t[(G_t - V(s_t))^2]$

Updates are performed using Adam optimizers with learning rates $\alpha_\pi = 10^{-3}$ and $\alpha_V = 10^{-3}$.

4.1.3. Downstream Model

The downstream classification model is a Convolutional Neural Network with architecture:

- Conv2D($1 \to 16$, 5×5) → ReLU → MaxPool(2×2)
- Conv2D($16 \to 32$, 5×5) → ReLU → MaxPool(2×2)
- Fully Connected($1568 \to 128$) → ReLU
- Fully Connected($128 \to 10$)

This model is trained on MNIST with cross-entropy loss and serves dual purposes:

1. Generating softmax predictions and entropy for state construction
2. Being the ultimate evaluation metric (validation accuracy after pseudo-labeling)

4.2. Approach 2: RLGSSL Framework

Inspired by Heidari et al. [1], we implemented a complete RLGSSL pipeline with novel adaptive enhancements.

4.2.1. Teacher-Student Framework

The teacher model θ_T maintains an exponential moving average (EMA) of the student model θ_S:

$$\theta_T \leftarrow \beta\theta_T + (1-\beta)\theta_S \tag{8}$$

with decay parameter $\beta = 0.999$. The teacher provides stable pseudo-labels $\tilde{y}_i^u = P_{\theta_T}(x_i^u)$ for unlabeled samples.

4.2.2. Mixup Data Generation

For each unlabeled sample x_i^u with teacher pseudo-label $\tilde{y}_i^u$, we sample a labeled example (x_j^l, y_j^l) and create a synthetic mixup sample:

$$\mu \sim \text{Beta}(1.0, 1.0) \tag{9}$$

$$x_i^m = \mu x_i^u + (1-\mu)x_j^l \tag{10}$$

$$y_i^m = \mu\tilde{y}_i^u + (1-\mu)\text{one-hot}(y_j^l) \tag{11}$$

This generates continuous interpolations in both feature and label spaces, with μ determining the mixing ratio.

4.2.3. RL Reward Function

The reward signal evaluates how well the student model performs on mixup data:

$$R(s, a; \text{sg}[\theta]) = -\text{MSE}(P_\theta(x_i^m), y_i^m) \tag{12}$$

where sg$[\cdot]$ denotes stop-gradient operation to prevent direct gradient flow through the model parameters during reward computation.

Adaptive Enhancements: We extend the base reward with:

$$R_{\text{adaptive}} = R_{\text{base}} + \lambda_{\text{conf}} \cdot \mathbb{E}[P_\theta(x_i^m)_{\max} \cdot y_i^m] \tag{13}$$

$$+ \lambda_{\text{div}} \cdot H(\mathbb{E}_{\text{batch}}[P_\theta(x^m)]) \tag{14}$$

$$+ \lambda_{\text{mix}} \cdot \text{Acc}_{\text{mixup}} \cdot (1 + \sigma(\mu)) \tag{15}$$

where:

- Confidence bonus ($\lambda_{\text{conf}} = 0.1$) rewards high-confidence predictions on high-confidence targets
- Diversity bonus ($\lambda_{\text{div}} = 0.05$) encourages diverse predictions across the batch
- Mixup effectiveness bonus weights performance by μ diversity

4.2.4. RL Loss Function

The RL loss combines KL divergence weighting with reward scaling:

$$\mathcal{L}_{\text{rl}} = -\mathbb{E}_{y_i^u \sim \pi_\theta}[\text{KL}(e, y_i^u) \times R(s, a; \text{sg}[\theta])] \tag{16}$$

where e is a uniform baseline distribution over classes.

Adaptive Enhancements:

- **Curriculum Learning**: Gradually increase focus on high-confidence samples:

$$w_{\text{curriculum}} = \min\left(1.0, \frac{\text{epoch}}{\text{max_epochs} \times 0.5}\right) \tag{17}$$

- **Entropy Regularization**: Prevent overconfident predictions:

$$\mathcal{L}_{\text{rl}}^{\text{adaptive}} = \mathcal{L}_{\text{rl}} + \lambda_{\text{entropy}} H(\tilde{y}_i^u) \tag{18}$$

  with $\lambda_{\text{entropy}} = 0.01$

- **Confidence-Based Weighting**: Focus learning on reliable pseudo-labels:

$$w_i = \mathbb{1}[\max(\tilde{y}_i^u) > \tau] \cdot (1 + w_{\text{curriculum}}) \tag{19}$$

  with threshold $\tau = 0.8$

4.2.5. Combined Loss Function

The total training loss integrates supervised, consistency, and RL components:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{sup}} + \lambda_{\text{cons}} \mathcal{L}_{\text{cons}} + \lambda_{\text{rl}} \mathcal{L}_{\text{rl}}^{\text{adaptive}} \tag{20}$$

where:

- $\mathcal{L}_{\text{sup}} = \text{CrossEntropy}(P_\theta(x^l), y^l)$ on labeled data
- $\mathcal{L}_{\text{cons}} = \text{MSE}(P_\theta(x^u), P_{\theta_T}(x^u))$ enforces student-teacher consistency
- Default weights: $\lambda_{\text{cons}} = 0.1$, $\lambda_{\text{rl}} = 0.1$

4.3. Implementation Details

Framework: PyTorch 2.0 with CUDA support

Datasets:

- Policy Gradient approach: MNIST (1000 labeled, 4000 unlabeled, 1000 validation)
- RLGSSL approach: CIFAR-10 (1000/2000/4000 labeled settings)

Training Configuration:

- Policy Gradient: 50 episodes, batch size 32, $\gamma = 0.99$
- RLGSSL: 50 warmup epochs + 400 RL epochs, batch size 128, learning rate 10^{-3}

Computational Environment: [TODO: Specify exact hardware - CPU vs GPU, RAM, etc.]

Code Repository: [TODO: Add GitHub repository link]

[TODO: Add algorithm pseudocode boxes for main training loops]

[TODO: Add more detailed architecture diagrams showing teacher-student interactions, mixup generation, and gradient flow]

5. Results and Discussion

This section presents our experimental results, including training dynamics, performance metrics, and analysis of the challenges encountered during implementation.

5.1. Experimental Setup

5.1.1. Dataset Configuration

Policy Gradient Experiments (MNIST):

- Total Training Samples: 60,000
- Labeled Samples: 1,000 (1.67%)
- Unlabeled Samples: 4,000
- Validation Samples: 1,000
- Test Samples: 10,000

RLGSSL Experiments (CIFAR-10):

- Total Training Samples: 50,000
- Labeled Samples: 1000/2000/4000 (2%/4%/8%)
- Unlabeled Samples: 49,000/48,000/46,000
- Test Samples: 10,000

5.1.2. Model Architectures

Policy Gradient Downstream Model:

Layer	**Output Shape**	**Parameters**
Conv2D (1→16, 5×5)	16×28×28	416
MaxPool2D (2×2)	16×14×14	0
Conv2D (16→32, 5×5)	32×14×14	12,832
MaxPool2D (2×2)	32×7×7	0
FC (1568→128)	128	200,832
FC (128→10)	10	1,290
Total		**215,370**

Table 1: Architecture of the downstream CNN model for MNIST classification

RLGSSL Model:

- Architecture: CNN-13 **[TODO: Add detailed layer breakdown]**
- Total Parameters: 3,567,370
- Trainable Parameters: 3,567,370

5.1.3. Hyperparameters

Parameter	Policy Gradient	RLGSSL
Learning Rate	10^{-3}	10^{-3}
Batch Size	32	128
Episodes/Epochs	50	50+400
Discount Factor (γ)	0.99	N/A
EMA Decay (β)	N/A	0.999
Mixup Alpha (α)	N/A	1.0
λ_{sup}	N/A	1.0
λ_{cons}	N/A	0.1
λ_{rl}	N/A	0.1
Confidence Threshold	N/A	0.8
Confidence Weight	N/A	0.1
Diversity Weight	N/A	0.05
Entropy Regularization	N/A	0.01

Table 2: Hyperparameter configurations for both approaches

5.2. Policy Gradient Results

[TODO: Add table with episode-by-episode results showing:

- Episode number
- Baseline accuracy
- New accuracy after pseudo-labeling
- Accuracy improvement
- Error rate
- Policy loss
- Value loss
- Average episode reward

]

[TODO: Add learning curves showing:

- Validation accuracy vs. episode
- Policy and value losses vs. episode
- Average reward vs. episode
- Error rate vs. episode

]

Key Observations:

- Initial baseline accuracy: ∼85% (trained on 1000 labeled MNIST samples)
- **[TODO: Report final best accuracy achieved]**
- **[TODO: Report average accuracy improvement per episode]**
- **[TODO: Analyze pseudo-labeling statistics: how many samples were pseudo-labeled vs. skipped]**

5.3. RLGSSL Results

5.3.1. Training Progression

Table 3 shows the training dynamics observed during RLGSSL experiments on CIFAR-10 with 1000 labeled samples.

Epoch	Acc (%)	Loss	RL Loss	Reward	Notes
5 (warmup)	29.12	–	–	–	Student: 29.12%, Teacher: 10.00%
60	36.51	0.2101	0.0102	-0.0298	Stable initial training
120	43.06	0.1808	-0.0018	-0.0298	Steady improvement
175	**45.55**	0.1285	-0.0383	-0.0294	Best accuracy before issues
180	44.98	0.1180	-0.0459	-0.0292	System hanging
240	36.20	-0.0485	-0.2837	-0.0342	**System crashed**

Table 3: RLGSSL training progression on CIFAR-10 (1000 labels). System crashed at epoch ∼275 due to stability issues with adaptive reward enhancements.

Performance Metrics:

- **Best Student Accuracy:** 45.55% (Epoch 175)
- **Training Time:** ∼15 hours before system crash
- **Final Status:** Incomplete due to computational instability

[TODO: Re-run complete experiments on GPU instances with:

- Comparison of base RLGSSL vs. adaptive enhancements
- Ablation studies on individual enhancement components
- Extended training to full 450 epochs
- Multiple random seeds for statistical significance

]

5.3.2. Comparison with Literature

Dataset	Labels	Architecture	Paper Result	Our Result
CIFAR-10	1000	CNN-13	9.15% error	**[Incomplete]**
CIFAR-10	2000	CNN-13	7.18% error	**[TODO]**
CIFAR-10	4000	CNN-13	6.11% error	**[TODO]**

Table 4: Comparison with Heidari et al. [1] results. Paper reports test error rates; equivalent accuracies are 90.85%, 92.82%, and 93.89% respectively.

Analysis Gap: Our best observed accuracy of 45.55% is significantly below the paper's reported 90.85% (9.15% error). This discrepancy likely stems from:

- Incomplete training (crashed at epoch 240 vs. target 450)
- Potential bugs in adaptive enhancement implementations
- Need for more careful hyperparameter tuning
- Possible differences in data preprocessing or augmentation strategies

[TODO: Complete full training runs and perform detailed error analysis]

5.4. Ablation Studies

[TODO: Conduct and report ablation studies on:

1. **Reward Function Components:** Compare base MSE reward vs. adding confidence/diversity bonuses individually
2. **Curriculum Learning:** Training with and without curriculum weighting
3. **Entropy Regularization:** Impact of entropy regularization weight
4. **Confidence Threshold:** Vary $\tau \in \{0.7, 0.8, 0.9, 0.95\}$
5. **EMA Decay:** Test $\beta \in \{0.99, 0.995, 0.999\}$

]

5.5. Computational Analysis

Training Time Breakdown (Estimated):

- Data loading and preprocessing: $\sim$2%
- Forward passes (student + teacher): $\sim$40%
- Mixup generation: $\sim$10%
- Backward passes and optimization: $\sim$45%
- Logging and evaluation: $\sim$3%

Memory Requirements:

- Model parameters (student + teacher): $\sim$28 MB
- Batch activations (batch size 128): $\sim$**[TODO: Calculate]**
- Gradient buffers: $\sim$**[TODO: Calculate]**

Scalability Concerns:

- Policy Gradient approach requires full episode completion before updates
- RLGSSL approach scales better due to mini-batch training
- Both approaches face challenges with very large datasets (>1M samples)

5.6. Analysis of Results

5.6.1. Strengths Observed

1. **Adaptive Behavior:** The Policy Gradient agent learned to balance pseudo-labeling and skipping, showing adaptive decision-making.
2. **Stable Initial Training:** RLGSSL showed stable improvements from epoch 60 to 175, demonstrating the potential of the approach.
3. **Framework Completeness:** Successfully implemented full pipelines including custom environments, RL agents, and downstream models.

5.6.2. Limitations and Challenges

1. **System Instability:** RLGSSL training crashed around epoch 275, likely due to numerical instability in adaptive reward computations or gradient explosions.
2. **Computational Cost:** Training on CPU for 15 hours before crash indicates need for GPU acceleration for practical applications.
3. **Performance Gap:** Results significantly below state-of-the-art, suggesting need for implementation debugging and refinement.
4. **Incomplete Evaluation:** Lack of complete training runs prevents definitive conclusions about approach effectiveness.

[TODO: Add visualization of prediction confidence distributions over training epochs]
[TODO: Add t-SNE or UMAP embeddings showing pseudo-labeled vs. labeled sample distributions]

6. Discussion

This section reflects on the key challenges encountered, lessons learned, and insights gained throughout the project implementation.

6.1. Technical Challenges

6.1.1. Reward Function Design

Designing effective reward functions proved to be one of the most challenging aspects:

- **Sparse vs. Dense Rewards:** Initial experiments with only end-of-episode accuracy rewards provided insufficient learning signal. Adding per-step confidence-based rewards improved learning but introduced new challenges.
- **Reward Scaling:** Balancing the magnitude of step rewards (confidence values in [0,1]) with episode-end accuracy bonuses (potentially large) required careful tuning.
- **Stability Issues:** The adaptive reward enhancements in RLGSSL introduced numerical instability. The diversity bonus, in particular, may have caused gradient explosions when batch entropy varied significantly.

Recommendation: Future work should implement reward clipping and normalization strategies used in modern RL algorithms like PPO.

6.1.2. Computational Constraints

The computational demands of RL-based pseudo-labeling present significant practical barriers:

- **Episode Length:** Each Policy Gradient episode processes all 4000 unlabeled samples, resulting in lengthy episodes and delayed learning.
- **CPU Limitations:** Training on CPU for 15+ hours without completion highlights the need for GPU acceleration. However, even with GPUs, the approach may struggle to scale to ImageNet-scale datasets.
- **Memory Requirements:** Maintaining teacher and student models simultaneously, along with mixup sample generation, doubles memory requirements compared to standard training.

Recommendation: Investigate mini-batch RL approaches and gradient accumulation strategies to improve computational efficiency.

6.1.3. Non-Stationarity and Credit Assignment

The pseudo-labeling environment violates key RL assumptions:

- **Moving Target:** The downstream model evolves during training, causing state distributions to shift continuously. This non-stationarity challenges standard RL convergence guarantees.
- **Delayed Feedback:** Accuracy improvements may not manifest until many episodes later, making credit assignment difficult. Which pseudo-labels at which training iterations contributed to final performance?
- **Exploration-Exploitation:** Balancing exploration of uncertain samples versus exploitation of confident predictions is complicated by the fact that exploration today affects future model capabilities.

Recommendation: Consider meta-learning approaches where the policy is trained across multiple dataset/model combinations to develop more robust pseudo-labeling strategies.

6.2. Insights on RLGSSL Formulation

Through implementation, we gained several insights into the RLGSSL approach:

6.2.1. Policy Network as Classifier

The elegant formulation where the classifier itself serves as the RL policy eliminates architectural complexity but introduces subtleties:

- **Gradient Flow:** Careful use of stop-gradient operations is crucial to prevent the model from "hacking" rewards by simply memorizing mixup samples.
- **Loss Weighting:** The relative weights of supervised, consistency, and RL losses dramatically affect behavior. In our experiments, RL loss dominated when set too high, causing instability.
- **Curriculum Necessity:** The curriculum learning component is not just an enhancement but appears necessary for stability, gradually introducing RL objectives as the model matures.

6.2.2. Mixup as Environment Dynamics

Using mixup to create the "environment" for RL is creative but has implications:

- **Sample Efficiency:** Mixup provides infinite synthetic samples, enabling dense RL learning signals.
- **Smoothness:** The continuous interpolation in mixup may help smooth the reward landscape, potentially aiding RL optimization.
- **Generalization:** Training on mixup samples may improve interpolation behavior, though extrapolation (out-of-distribution samples) remains challenging.

6.3. Comparison: Policy Gradient vs. RLGSSL

Table 5 compares the two approaches implemented:

Aspect	Policy Gradient	RLGSSL
Complexity	Higher: separate RL agent + downstream model	Lower: unified architecture
Scalability	Poor: full episode required	Better: mini-batch training
Interpretability	Higher: explicit state-action mapping	Lower: implicit RL in loss function
Flexibility	Higher: easy to modify reward/state	Lower: coupled to mixup strategy
Performance	**[TODO: report]**	**[TODO: report]**
Stability	**[TODO: report]**	Issues with adaptive enhancements

Table 5: Comparison of the two RL-based pseudo-labeling approaches

6.4. Practical Considerations

6.4.1. When to Use RL for Pseudo-Labeling

Based on our experience, RL-based pseudo-labeling is most appropriate when:

- Labeled data is extremely limited (<1% of dataset)
- Pseudo-labeling decisions have long-term consequences (e.g., medical diagnosis)
- Computational resources are available for extensive training
- The problem exhibits clear reward signals (e.g., validation accuracy)

It may be overkill when:

- Labeled data is moderately available (>10% of dataset)
- Simple confidence thresholding already works well
- Real-time or resource-constrained deployment is required
- The dataset is extremely large (millions of samples)

6.4.2. Engineering Recommendations

From an engineering perspective, several practices proved valuable:

1. **Modular Design:** Separating environment, agent, and downstream model into distinct modules facilitated debugging and experimentation.
2. **Logging Infrastructure:** Comprehensive logging with TensorBoard was essential for diagnosing training dynamics and stability issues.
3. **Checkpointing:** Regular model checkpointing allowed recovery from crashes and comparison of models at different training stages.
4. **Ablation-Friendly Code:** Designing hyperparameters and enhancement flags in a central config file enabled rapid ablation studies.

6.5. Future Directions

Several promising directions for future work emerged:

1. **Stability Improvements:** Implement gradient clipping, reward normalization, and PPO-style trust region constraints to address training instability.
2. **Multi-Objective RL:** Explicitly model multiple objectives (accuracy, diversity, computational cost) using multi-objective RL frameworks.
3. **Meta-Learning:** Train pseudo-labeling policies across multiple datasets to develop generalizable strategies.
4. **Uncertainty Quantification:** Integrate Bayesian deep learning or ensemble methods to provide better uncertainty estimates for state representations.
5. **Hierarchical RL:** Use hierarchical RL to separate high-level strategy (which data to focus on) from low-level tactics (specific label assignments).
6. **Offline RL:** Pre-collect trajectories with simple baseline policies, then use offline RL to learn from these without expensive online interactions.

6.6. Limitations of Current Work

We acknowledge several limitations:

- **Incomplete Experiments:** System crashes prevented complete training runs and comprehensive evaluation.
- **Limited Datasets:** Experiments focused on MNIST and CIFAR-10; generalization to other domains (text, speech, medical imaging) remains unexplored.
- **Hyperparameter Sensitivity:** Limited computational budget prevented extensive hyperparameter searches.
- **Comparison Baselines:** Lacking direct comparisons with strong SSL baselines like FixMatch, MixMatch, or simple confidence thresholding.
- **Theoretical Analysis:** No formal convergence guarantees or theoretical characterization of when RL-based pseudo-labeling should outperform alternatives.

[TODO: Add a subsection comparing results with standard SSL baselines (FixMatch, MixMatch, simple pseudo-labeling with confidence threshold)]

7. Conclusion

This capstone project investigated the application of reinforcement learning to intelligent pseudo-labeling in semi-supervised learning contexts. We developed and evaluated two complementary approaches: a custom Policy Gradient framework with explicit state-action formulations, and an implementation of the novel RLGSSL approach with adaptive enhancements.

7.1. Key Contributions

Our work makes several contributions:

1. **Complete RL Framework:** Developed a full-featured custom OpenAI Gymnasium environment for pseudo-labeling, including carefully designed state spaces (image features + model predictions + entropy), action spaces (label assignment or skip), and confidence-weighted reward functions.

2. **RLGSSL Implementation:** Successfully implemented the RLGSSL framework proposed by Heidari et al., including teacher-student architecture, mixup augmentation, and the novel RL reward formulation.

3. **Adaptive Enhancements:** Extended RLGSSL with confidence bonuses, diversity regularization, curriculum learning, and entropy regularization mechanisms not present in the original paper.

4. **Practical Insights:** Documented computational challenges, stability issues, and engineering considerations crucial for practitioners attempting to apply RL to pseudo-labeling.

5. **Open-Source Implementation: [TODO: Add GitHub repository link]** Provided well-documented, modular code for future research and experimentation.

7.2. Main Findings

Technical Feasibility: RL-based pseudo-labeling is technically feasible and can be implemented using standard deep RL techniques. However, significant engineering effort is required to achieve stability and scalability.

Computational Demands: Both approaches exhibit substantial computational requirements, with training times exceeding 15 hours on CPU for moderate-scale experiments. GPU acceleration and algorithmic optimizations are essential for practical deployment.

Stability Challenges: Adaptive reward enhancements, while theoretically motivated, introduced numerical instability that caused training crashes. Careful implementation with gradient clipping and reward normalization is crucial.

Performance Gap: Our results showed a significant gap compared to state-of-the-art SSL methods, likely due to incomplete training, implementation bugs, and insufficient hyperparameter tuning. This highlights the challenge of reproducing research results and the importance of careful validation.

7.3. Lessons Learned

Reward Engineering is Critical: The choice and design of reward functions dramatically affects learning dynamics. Simple confidence-based rewards worked reasonably well, but complex multi-component rewards require careful balancing and stability mechanisms.

Start Simple, Add Complexity Gradually: Our experience suggests starting with minimal RL formulations and incrementally adding enhancements while rigorously testing stability at each stage.

Computational Planning is Essential: RL-based approaches require significantly more computation than standard SSL. Projects should budget computational resources accordingly and prioritize access to GPU infrastructure.

Reproducibility is Challenging: Even with detailed papers, reproducing research results requires careful attention to implementation details, data preprocessing, and hyperparameter settings that may not be fully documented.

7.4. Broader Impact

This work contributes to the growing body of research applying reinforcement learning to data-centric AI problems. As labeled data remains a bottleneck in many domains—especially in medical imaging, autonomous systems, and low-resource languages—intelligent pseudo-labeling strategies could significantly reduce annotation costs and democratize access to machine learning.

However, practitioners should be aware of the computational costs and implementation complexity. For many applications, simpler SSL baselines (FixMatch, MixMatch) may provide better cost-benefit trade-offs. RL-based approaches are most valuable when:

- Labeled data is extremely scarce (<1% of dataset)
- Label quality is critical (high-stakes domains)
- Computational resources are available
- The problem exhibits clear, measurable objectives

7.5. Future Work

To build upon this foundation, future research should:

1. **Complete Experimental Evaluation:** Conduct full training runs on GPU infrastructure with multiple random seeds for statistical significance.
2. **Stability Improvements:** Implement PPO-style clipping, gradient normalization, and reward preprocessing to improve training stability.
3. **Comprehensive Baselines:** Compare directly with FixMatch, MixMatch, Mean Teacher, and simple confidence thresholding across multiple datasets.
4. **Ablation Studies:** Systematically evaluate each component (curriculum learning, entropy regularization, confidence bonuses) to understand their individual contributions.
5. **Scalability Analysis:** Investigate mini-batch RL methods and gradient accumulation strategies to enable scaling to larger datasets.
6. **Domain Transfer:** Extend experiments to other domains (text classification, medical imaging, speech recognition) to assess generalizability.
7. **Theoretical Analysis:** Develop formal characterizations of when RL-based pseudo-labeling should outperform heuristic approaches.
8. **Meta-Learning:** Train policies across multiple dataset/architecture combinations to develop generalizable pseudo-labeling strategies.

7.6. Closing Remarks

This capstone project demonstrated both the promise and challenges of applying reinforcement learning to semi-supervised learning. While we encountered significant obstacles—training instability, computational constraints, and performance gaps—the experience provided valuable insights into the complexities of RL-based data-centric AI.

The field of RL-guided semi-supervised learning remains in its early stages, with much room for innovation. As computational resources become more accessible and RL algorithms continue to improve, intelligent pseudo-labeling may become a standard tool in the machine learning practitioner's toolkit.

We hope this work serves as a foundation for future researchers and provides both inspiration and cautionary lessons for those embarking on similar projects.

References

[1] A. Heidari, A. Gheibi, I. Ullah, and N. Salimi, "Reinforcement learning-guided semi-supervised learning," *arXiv preprint arXiv:2405.01760v1*, 2024. [Online]. Available: https://arxiv.org/abs/2405.01760

A. Appendix

A.1. A: Code Structure

The project codebase is organized as follows:

```
Capstone/
 src/
    component/
       main.py           # Main training loop (Policy Gradient)
       model.py          # Policy and Value Networks
       env.py            # Custom PseudoLabelEnv
       utils.py          # Helper functions
       config.py         # Hyperparameter configuration
    requirements.txt      # Python dependencies
 reports/
    RLGSSL_Report_Draft.md     # RLGSSL implementation notes
    Progress_Report/
       Progress_Report.md     # Weekly progress tracking
    Latex_report/
        Final_Report.tex       # This document
        Sample_Report.tex      # Template
        File_Setup.tex         # LaTeX configuration
        references.bib         # Bibliography
        fig/                   # Figures directory
 README.md                      # Project overview
```

A.2. B: Hyperparameter Configuration

Complete hyperparameter settings used in experiments:

Policy Gradient (config.py):

```python
# Training parameters
EPISODES = 50
NUM_LABELED = 1000
NUM_UNLABELED = 4000
NUM_VALIDATION = 1000

# Policy Gradient parameters
GAMMA = 0.99
ALPHA_PI = 1e-3
ALPHA_V = 1e-3
HIDDEN_DIM = 64
POLICY_TYPE = 'mlp'

# Downstream Model parameters
DOWNSTREAM_LR = 0.001
DOWNSTREAM_EPOCHS = 1
BATCH_SIZE = 32
```

RLGSSL (from draft documentation):

```python
# Training Configuration
WARMUP_EPOCHS = 50
RLGSSL_EPOCHS = 400
BATCH_SIZE = 128
LEARNING_RATE = 0.001
```

```
# Teacher-Student
EMA_DECAY = 0.999

# Mixup
MIXUP_ALPHA = 1.0

# Loss Weights
LAMBDA_SUP = 1.0
LAMBDA_CONS = 0.1
LAMBDA_RL = 0.1

# Adaptive Enhancements
CONFIDENCE_THRESHOLD = 0.8
CONFIDENCE_WEIGHT = 0.1
DIVERSITY_WEIGHT = 0.05
ENTROPY_REGULARIZATION = 0.01
```

A.3. C: Key Implementation Details

EMA Teacher Update:

```
def update_ema_teacher(student_model, teacher_model, beta=0.999):
    for teacher_param, student_param in zip(
        teacher_model.parameters(),
        student_model.parameters()
    ):
        teacher_param.data = (
            beta * teacher_param.data +
            (1 - beta) * student_param.data
        )
```

Mixup Generation:

```
def generate_mixup(unlabeled_batch, labeled_batch, alpha=1.0):
    mu = np.random.beta(alpha, alpha)
    x_u, pseudo_y_u = unlabeled_batch
    x_l, y_l = labeled_batch

    # Create mixup samples
    x_m = mu * x_u + (1 - mu) * x_l
    y_m = mu * pseudo_y_u + (1 - mu) * F.one_hot(y_l)

    return x_m, y_m, mu
```

Confidence-Based Reward:

```
def calculate_reward(pred_label, true_label, action, probs):
    if action == num_classes:  # skip action
        return -0.1
    confidence = probs[action].item()
    if action == true_label:
        return confidence  # positive reward for correct
    else:
        return -confidence  # negative reward for incorrect
```

A.4. D: References for Additional Reading

Reinforcement Learning Fundamentals:

- Sutton & Barto (2018), "Reinforcement Learning: An Introduction" (2nd Edition)
- Schulman et al. (2017), "Proximal Policy Optimization Algorithms"
- Mnih et al. (2016), "Asynchronous Methods for Deep Reinforcement Learning"

Semi-Supervised Learning:

- **[TODO: Add MixMatch citation (Berthelot et al., 2019)]**
- **[TODO: Add FixMatch citation (Sohn et al., 2020)]**
- **[TODO: Add Mean Teacher citation (Tarvainen & Valpola, 2017)]**

OpenAI Gymnasium:

- Official Documentation: https://gymnasium.farama.org/
- Custom Environment Tutorial: https://gymnasium.farama.org/tutorials/gymnasium_basics/environment_creation/

A.5. E: Acknowledgments

[TODO: Add acknowledgments for:

- Prof. Tyler Wallett for project guidance and proposal
- Dr. Amir Jafari for supervision
- Computing resources (if applicable)
- Collaborators (if applicable)
- Open-source libraries used (PyTorch, Gymnasium, etc.)

]